

File Transfer Agent

Comprehensive Technical Documentation & Architectural Reference Manual

1. Executive Summary

The **File Transfer Agent** is a modern, full-stack desktop & network application designed for high-performance, low-latency file and directory transfers across local filesystems and local area networks (LAN). Built with a decoupled architecture comprising a React frontend and a Node.js backend agent, the system enables users to copy, move, monitor, and manage files through an intuitive web interface backed by full-duplex WebSocket streaming.

Parameter	Specification / Technology
Application Type	Client-Server Local File Management Agent
Frontend Framework	React 18 + TypeScript + Vite + Tailwind CSS
Backend Agent Runtime	Node.js (v20+) + Express + WebSocket (ws)
Real-Time Protocol	WebSockets (Port 3456) with bi-directional RPC
HTTP Server API	Express.js (Port 3457) for REST & System Info
OS Compatibility	Windows 10/11, macOS, Linux (Cross-Platform)
Authentication	SHA-256 Token-Based Auth (`7fca426f...`)

2. System Architecture & Components

The application relies on a strict separation of concerns between the presentation layer (Frontend) and the execution engine (Agent Server).

A. Frontend Client (`frontend/`)

- React 18 Component Hierarchy:** Built with modular components including `TransferPanel`, `BrowseModal`, `HistoryPanel`, `SettingsPanel`, and `ConnectionStatus`.
- State Management & Custom Hooks:** Powered by `useTransfers` (handles live transfer queues and event streams), `useBrowse` (manages directory tree exploration), and `useWebSocket` (manages persistent connections & auto-reconnects).
- Tailwind CSS UI:** Responsive design featuring dark/light theme options, speed meters, ETA indicators, and interactive modal dialogs.

B. Agent Backend (`agent/`)

- WebSocket Server (`ws`):** Listens on port 3456 to handle real-time RPC messages including `auth`, `transfer`, `pause`, `resume`, `cancel`, `browse`, and `settings`.
- Stream Engine (`services/transfer.ts`):** Uses Node `fs.createReadStream` and `createWriteStream` with configurable chunk sizes (32KB–64KB) to stream data with minimal memory consumption.

- **Disk & Drive Discovery** (`services/browse.ts`): Utilizes PowerShell `Get-CimInstance Win32_LogicalDisk` on Windows and standard POSIX system utilities (`df -h`) on Linux/macOS.
- **Input Sanitizer** (`utils/fs.ts`): Strips quotes from Windows path copies and normalizes path separators to prevent security traversal and runtime errors.

3. Key Features & Capabilities

- **High-Speed File & Folder Operations:** Supports both single file and recursive directory Copy and Move (cut-and-paste) operations across local and external drives.
- **Real-Time Progress & Analytics:** Calculates transfer speed (KB/s or MB/s), remaining time (ETA), percentage completed, and current active file name.
- **Conflict Resolution Strategies:** Offers three configurable modes when destination files exist: *Rename (keep both with counter)*, *Overwrite*, or *Skip*.
- **Cross-Device Access:** Allows any device on the local network (smartphones, tablets, secondary laptops) to open `http://:5173` and control transfers remotely.
- **Visual Directory Picker:** Interactive modal browsing allowing drive selection, folder navigation, and direct file picking.

4. WebSocket API Protocol Reference

Communication between client and backend agent uses structured JSON messages:

Message Type	Direction	Description & Payload
<code>`auth`</code>	Client → Agent	Authenticates session via token string.
<code>`transfer`</code>	Client → Agent	Initiates copy/move. Payload: <code>`sourcePath`</code> , <code>`destinationPath`</code> , <code>`operation`</code> , <code>`conflictResolution`</code> .
<code>`progress`</code>	Agent → Client	Broadcasts live byte counts, speed, and ETA.
<code>`pause`</code> / <code>`resume`</code>	Client → Agent	Pauses or resumes an active transfer task by ID.
<code>`cancel`</code>	Client → Agent	Aborts transfer and cleans up open file streams.
<code>`drives`</code> / <code>`browse`</code>	Client → Agent	Fetches drive list or inspects folder contents.

5. Deployment & Execution Guide

To start the File Transfer Agent ecosystem:

1. Install Dependencies:

```
npm run install:all
```

2. Build Agent & Frontend Services:

```
npm run build
```

3. Start Background Agent & Web Interface:

```
npm start
```

Once running, access the web console at **http://localhost:5173** or on LAN at **http://192.168.29.154:5173**.